

DAX Formulae for Power BI

1. COUNT

Counts only **numeric non-blank values** in a column.

```
COUNT (Table[Column])
```

Example:

```
Total_Customers = COUNT (Sales[CustomerID])
```

Use When:

- Column contains numbers
- You want to ignore blanks and text

Important:

- Does NOT count text
- Does NOT count blanks

2. COUNTA

Counts **all non-blank values** (text, numbers, boolean)

```
COUNTA (Table[Column])
```

Example:

```
Total_Entries = COUNTA (Sales[CustomerName])
```

Use When:

- Column has mixed data (text + numbers)

Important:

- Counts text
- Counts numbers
- Counts TRUE/FALSE
- Ignores blanks

3. COUNTBLANK

Counts only **blank (empty) values**

```
COUNTBLANK (Table[Column])
```

Example:

```
Missing_Values = COUNTBLANK(Sales[CustomerName])
```

Use When:

- You want to find missing data

4. COUNTROWS

Counts total **number of rows** in a table

```
COUNTROWS(Table)
```

Example:

```
Total_Rows = COUNTROWS(Sales)
```

Use When:

- You want total records (rows)

With Filter:

```
UP_Records = COUNTROWS(FILTER(Sales, Sales[State] = "UP"))
```

5. DISTINCTCOUNT

Counts **unique (distinct) non-blank values**

```
DISTINCTCOUNT(Table[Column])
```

Example:

```
Unique_Customers = DISTINCTCOUNT(Sales[CustomerID])
```

Use When:

- You want unique values count

Important:

- Does NOT count blanks

6. DISTINCTCOUNTNOBLANK

Counts **unique values but excludes blanks explicitly**

```
DISTINCTCOUNTNOBLANK(Table[Column])
```

Example:

```
Unique_Customers_NoBlank = DISTINCTCOUNTNOBLANK(Sales[CustomerID])
```

Use When:

- You want only real unique values (no blank considered)

Note:

- Similar to DISTINCTCOUNT, but safer when blanks exist

7. COUNTX

Counts values **after evaluating an expression row-by-row**

```
COUNTX (Table, Expression)
```

Example:

```
High_Rating_Count =  
COUNTX (  
    Sales,  
    IF (Sales[Rating] >= 4, 1, BLANK())  
)
```

Use When:

- You need **condition-based counting**
- Works like loop (row-by-row)

Important:

- Works on expressions
- More flexible than COUNT

QUICK COMPARISON TABLE

Function	Counts What	Ignores Blanks	Counts Text	Special Use
COUNT	Numbers	✓	✗	Numeric only
COUNTA	All values	✓	✓	Mixed data
COUNTBLANK	Only blanks	✗	✗	Missing data
COUNTROWS	Rows	✗	✓	Total records
DISTINCTCOUNT	Unique values	✓	✓	Unique count
DISTINCTCOUNTNOBLANK	Unique (no blank)	✓	✓	Clean unique
COUNTX	Expression result	✓	✓	Conditional

1. SUM (Basic Aggregation)

Adds all values of a **single column**

```
SUM(Table[Column])
```

Example:

```
Total_Sales = SUM(Sales[Amount])
```

Explanation:

It simply adds all values in the *Amount* column.

Use When:

- You just need **simple total**
- No row-by-row logic needed

Key Point:

- Fast
- Works only on a column
- No custom logic inside

2. SUMX (Row-by-Row Calculation)

Performs calculation **for each row**, then sums the result

```
SUMX(Table, Expression)
```

Example:

```
Total_Sales = SUMX(Sales, Sales[Quantity] * Sales[Price])
```

Explanation:

- First: multiply Quantity $\times$ Price for each row
- Then: add all results

Use When:

- You need **calculation before sum**
- Example: profit, total price, etc.

Key Point:

- Flexible
- Slower than SUM (because row-by-row)

3. CALCULATE

Changes the **filter context** and then performs calculation

```
CALCULATE(Expression, Filter)
```

Example:

```
UP_Sales = CALCULATE(SUM(Sales[Amount]), Sales[State] = "UP")
```

Explanation:

- First applies filter → State = "UP"
- Then calculates SUM of Amount

Use When:

- You want **filtered results**
- Dynamic calculations in reports

Key Point:

- Most powerful DAX function
- Used in almost every dashboard

DIFFERENCE

Feature	SUM	SUMX	CALCULATE
Type	Simple	Iterator	Filter Modifier
Works On	Column	Table + Expression	Expression
Row-by-row	✗	✓	✗
Filters	✗	✗	✓
Use Case	Simple total	Custom calculation	Filtered result

SIMPLE UNDERSTANDING

- SUM → “Just add”
- SUMX → “Calculate then add”
- CALCULATE → “Filter then calculate”

REAL EXAMPLE (INTERVIEW LEVEL)

Scenario: Total sales in UP

Using SUM (not possible directly)

```
SUM(Sales[Amount])  -- no filter
```

Using CALCULATE

```
UP_Sales = CALCULATE(SUM(Sales[Amount]), Sales[State] = "UP")
```

Custom Calendar – Power Query

1. List.Dates

Creates a **list of continuous dates** in Power Query (M language)

```
List.Dates(start_date, number_of_days, step)
```

Example:

```
= List.Dates(#date(2023,1,1), 365, #duration(1,0,0,0))
```

Explanation:

- Start → 1 Jan 2023
- 365 → total days
- Step → 1 day increment

Use:

- Creating custom date table in Power BI (Power Query)

2. CALENDAR (Min–Max)

Creates a date table using **minimum and maximum dates from data**

Syntax:

```
CALENDAR(StartDate, EndDate)
```

Example:

```
DateTable = CALENDAR(MIN(Sales[Date]), MAX(Sales[Date]))
```

Use:

- When you want dynamic date range based on dataset

3. CALENDARAUTO

Automatically creates a date table by detecting **all date columns**

Syntax:

```
CALENDARAUTO()
```

Example:

```
DateTable = CALENDARAUTO()
```

Use:

- Fast and automatic date table creation

Note:

- Uses entire model (not just one table)

4. MTD, QTD, YTD

Time intelligence functions for cumulative totals

MTD (Month-To-Date)

```
Sales_MTD = TOTALMTD(SUM(Sales[Amount]), DateTable[Date])
```

From start of month → today

QTD (Quarter-To-Date)

```
Sales_QTD = TOTALQTD(SUM(Sales[Amount]), DateTable[Date])
```

From start of quarter → today

YTD (Year-To-Date)

```
Sales_YTD = TOTALYTD(SUM(Sales[Amount]), DateTable[Date])
```

From start of year → today

Requirement:

- Proper **Date Table** must exist

5. NETWORKDAYS

Counts **working days** between two dates (excluding weekends & holidays)

Syntax:

```
NETWORKDAYS(StartDate, EndDate)
```

Example:

```
Work_Days = NETWORKDAYS("01-01-2023", "31-01-2023")
```

6. DATESBETWEEN

Returns dates **between two specific dates**

Syntax:

```
DATESBETWEEN(DateColumn, StartDate, EndDate)
```

Example:

```
Sales_Jan =  
CALCULATE(  
    SUM(Sales[Amount]),  
    DATESBETWEEN(DateTable[Date], DATE(2023,1,1), DATE(2023,1,31))  
)
```

Use:

- Custom date filtering

TEXT FUNCTIONS

1. UPPER

Converts text to **uppercase**

```
UPPER("power bi")    -- "POWER BI"
```

2. LOWER

Converts text to **lowercase**

```
LOWER("POWER BI")    -- "power bi"
```

3. LEN

Returns **length of text**

```
LEN("Power BI")      -- 8
```

QUICK SUMMARY TABLE

Function	Purpose
List.Dates	Create date list (Power Query)
CALENDAR	Date table using min-max
CALENDARAUTO	Auto date table
MTD	Monthly total
QTD	Quarterly total

Function	Purpose
YTD	Yearly total
NETWORKDAYS	Working days
DATESINPERIOD	Dynamic period
DATESBETWEEN	Fixed range
UPPER	Uppercase
LOWER	Lowercase
LEN	Text length

Advanced DAX (Concept)

Core Idea:

Advanced DAX is all about **filter context + row context control**

Key Learning:

- CALCULATE = king
- Filters control everything
- Measures are dynamic

Important Line:

“DAX works based on context (filter context + row context)”

Create Calendar Table

Method 1: Using CALENDAR

```
DateTable =
CALENDAR(MIN(Sales[Date]), MAX(Sales[Date]))
```

Creates:

- Continuous date table
- Based on dataset

Method 2: CALENDARAUTO

```
DateTable = CALENDARAUTO()
```

Automatically:

- Detects min & max dates
- Scans full model

Add Columns (Important)

```
Year = YEAR(DateTable[Date])
Month = FORMAT(DateTable[Date], "MMM")
MonthNo = MONTH(DateTable[Date])
```

Use:

- Sorting month
- Time intelligence

DISTINCT DAX Formula

1. DISTINCT

Returns **unique values table**

```
Unique_States = DISTINCT(Sales[State])
```

2. DISTINCTCOUNT

```
Unique_Count = DISTINCTCOUNT(Sales[State])
```

Counts unique values

Use Case:

- Unique customers
- Unique products

3. RANKX Function

Ranks values based on expression

Syntax:

```
RANKX(Table, Expression, , Order, Type)
```

Example:

```
Rank_Sales =
RANKX(
    ALL(Sales[Product]),
    SUM(Sales[Amount]),
    ,
    DESC,
    DENSE
)
```

Explanation:

- ALL → removes filter
- SUM → value to rank

- DESC → highest = rank 1

Types:

- DENSE → 1,2,2,3
- SKIP → 1,2,2,4

CALCULATETABLE

Returns **table instead of single value**

Similar to CALCULATE but:

- CALCULATE → scalar value
- CALCULATETABLE → table

Syntax:

`CALCULATETABLE (Table, Filter)`

Example:

```
Top_Customers =
CALCULATETABLE (
    Sales,
    Sales[Amount] > 1000
)
```

Use Case:

- Create filtered tables
- Advanced analysis

FINAL SUMMARY (IMPORTANT)

Concept	Key Idea
CALCULATE	Modify filter context
KEEPFILTERS	Add filter (don't replace)
REMOVEFILTERS / ALL	Ignore filters
DISTINCT	Unique values
RANKX	Ranking
CALCULATETABLE	Returns table
Calendar Table	Base of time intelligence